

プロジェクト初期化 フェーズ

プロジェクト初期化 フェーズ

backend / frontend / DB の最小構成を立ち上げ、以降の開発の土台を作る

Hyonun Kin

2026-04-28

目的・背景

文章校正サービスの実装に先立ち、後続フェーズが破綻なく進むための土台を整える。この時点では機能要件は1つも実装しない。代わりに、「**実装が始まった瞬間に詰まらない**」状態を目指す。

具体的には、

- backend を起動できる (FastAPI / uv)
- frontend を起動できる (SvelteKit / bun)
- DB を起動できる (PostgreSQL / Docker Compose)
- テストが動く (pytest)
- バージョン管理が始まっている (git, develop ブランチ)

この5つを担保する。

要件

- 機能要件は無し (インフラ整備のみ)
- 非機能要件として以下を満たす
 - `uv run pytest` がスモークテストを通す
 - `bun run dev` が SvelteKit を起動する
 - `docker compose up -d` で PostgreSQL が立ち上がる
 - `.env` を使用しないと壊れる箇所は無い (phase 1 時点では DB / AI / JWT は未使用)

設計

リポジトリ構成

```
docs-checker-app/
├── backend/                # FastAPI + uv
│   ├── api/
│   │   ├── __init__.py
│   │   └── app.py          # /health (疎通確認用)
│   ├── prompts/
│   │   └── correct.yaml    # 校正プロンプトの雛形（中身は phase 3 で記述）
│   ├── docker/
│   │   └── compose.yml     # PostgreSQL 16
│   ├── tests/
│   │   ├── __init__.py
│   │   └── test_smoke.py   # /health の TestClient テスト
│   ├── .env.example
│   ├── pyproject.toml
│   └── uv.lock
├── frontend/              # SvelteKit (TypeScript, minimal) + bun
│   ├── src/
│   ├── package.json
│   ├── svelte.config.js
│   └── ... (sv create で生成)
├── docs/
│   └── phase-1.md          # 本ドキュメント
├── .gitignore
└── README.md
```

採用した依存

backend は最小限のみ追加した。phase 2 以降で順次追加する想定。

パッケージ	用途	フェーズ
fastapi	API フレームワーク	1
uvicorn[standard]	ASGI サーバー	1
httpx	テスト用 HTTP クライアント（TestClient が依存）	1
pytest (dev)	テスト	1

frontend はテンプレ最小（minimal + TypeScript）で生成し、Tailwind / shadcn-svelte は phase 4 で shadcn-svelte init 経由で追加する方針。

Docker Compose

PostgreSQL 16 を `docs-checker-db` コンテナで起動。

- ポート : `5432`
- ユーザー / パスワード / DB 名は環境変数 (デフォルト `app/app/docs_checker`)
- ボリューム `db-data` で永続化
- `pg_isready` で healthcheck

phase 1 時点ではまだ DB に接続するコードは無いため、起動できることのみを確認する。

実装

主要コード

backend のスモーク用エンドポイント (`backend/api/app.py`) :

```
from fastapi import FastAPI

app = FastAPI(title="Docs Checker API")

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}
```

スモークテスト (`backend/tests/test_smoke.py`) :

```
from fastapi.testclient import TestClient

from api.app import app

def test_health() -> None:
    client = TestClient(app)
    res = client.get("/health")
    assert res.status_code == 200
    assert res.json() == {"status": "ok"}
```

環境変数の雛形 (`.env.example`)

phase 2 以降で使うものも先に並べておく。値は空 or デフォルト。

```
GEMINI_API_KEY=
GEMINI_MODEL=gemini-2.5-flash-lite
DATABASE_URL=postgresql+asyncpg://app:app@localhost:5432/docs_checker
JWT_SECRET=
INITIAL_ADMIN_USERNAME=admin
INITIAL_ADMIN_PASSWORD=
```

検証

項目	結果
<code>uv sync</code>	通る
<code>uv run pytest -v</code>	<code>test_health</code> 1 件 pass
<code>bun install</code>	通る
<code>docker compose -f docker/compose.yml config</code>	構成 OK (実起動は未確認)

確認観点 (architecture.md より)

- 設計通りになっているか (architecture.md のディレクトリ構成と一致)
- 環境変数をハードコードしてない (`.env.example` のみ、実値は空)
- 自分で理解できるコードか (10行未満の最小コード)

次フェーズへの引き継ぎ

phase 2 (DB + 認証) で着手するもの：

- `backend/api/models.py` — `User` テーブル定義
- `backend/api/session.py` — `AsyncSession` の依存関数
- `backend/api/auth.py` — JWT 発行 / 検証、bcrypt ハッシュ
- `POST /api/login` / `POST /api/logout` の実装
- 起動時の admin 自動作成 (冪等：「無ければ作る」)
- 認証関連の pytest を追加

ブランチ運用は `feature/phase-2` を `develop` から切る。